



ASK 1: Language Translation Tool ● Create a user interface where user can enter text and select source & target languages. ● Use a translation API like Google Translate API or Microsoft Translator to process the input. ● Send the text to the API and get the translated response. ● Display the translated text clearly on the screen. ● Optional: Add a copy button or text-to-speech feature for better usability.

1) Install required libs

```
!pip install -q googletrans==4.0.0-rc1 ipywidgets gTTS
```

2) Imports

```
from googletrans import Translator, LANGUAGES
from ipywidgets import Textarea, Dropdown, Button, HBox, VBox, Label, Output
from IPython.display import display, Audio, clear_output
from gtts import gTTS
import io
```

3) Build UI

```
translator = Translator()
```

```
languages = {k:v for k,v in LANGUAGES.items()} # language code -> name
```

```
# build options for dropdown (code: name)
```

```
options = [(name.capitalize(), code) for code,name in languages.items()]
```

```
src_dropdown = Dropdown(options=[('Auto', 'auto')] + options, value='auto', d
```

```
tgt_dropdown = Dropdown(options=options, value='en', description='Target:')
```

```
input_box = Textarea(value='Hello, how are you?', description='Text:', layout=
```

```
translate_btn = Button(description='Translate', button_style='success')
```

```
tts_btn = Button(description='Play TTS', button_style='info')
```

```
copy_btn = Button(description='Copy', button_style='')
```

```
out = Output()
```

```
def do_translate(_):
```

```
    with out:
```

```
        clear_output()
```

```
        text = input_box.value.strip()
```

```
        if not text:
```

```
            print("Enter text to translate.")
```

```
            return
```

```
        src = src_dropdown.value
```

```
        tgt = tgt_dropdown.value
```

```
        print(f"Source language: {src}, Target language: {tgt}") # Added for c
```

```
        try:
```

```
            res = translator.translate(text, src=src, dest=tgt)
```

```
            print("Original (detected):", res.src, "\nOriginal text:", res.or
```

```
            print("\nTranslated text:", res.text)
```

```
            # store for TTS
```

```
            out.last_translation = res.text
```

```

        out.last_lang = res.dest
    except Exception as e:
        print("Translation error:", e)

def play_tts(_):
    with out:
        if not hasattr(out, 'last_translation'):
            print("Translate first.")
            return
        tts = gTTS(text=out.last_translation, lang=out.last_lang)
        bio = io.BytesIO()
        tts.write_to_fp(bio)
        bio.seek(0)
        display(Audio(bio.read(), autoplay=True))

def copy_text(_):
    # copy to clipboard is limited in notebooks; provide easy print to allow
    with out:
        if not hasattr(out, 'last_translation'):
            print("Translate first.")
            return
        print("\n=== Copy below ===\n")
        print(out.last_translation)

translate_btn.on_click(do_translate)
tts_btn.on_click(play_tts)
copy_btn.on_click(copy_text)

ui = VBox([src_dropdown, tgt_dropdown, input_box, HBox([translate_btn, tts_btn])])
display(ui)

# Notes:
# - googletrans is unofficial and may break if Google changes endpoints.
# - For production, use Google Cloud Translate API or Microsoft Translator (s

VBox(children=(Dropdown(description='Source:', options=((('Auto', 'auto'), ('A

```